CSAI2017P — Applied Machine Learning (Theory)
Theory Quiz 4 — Solutions • Set A

Maximum marks **20** • 15 questions on four pages

These are the answers to **Set A**, with the reasoning behind each one. Find the set letter on your own paper and work from the matching four pages. Sets were handed out at random and every set covers the same material, so if you want the extra practice the other sets are worth reading too.
Several of these questions have more than one good route to the answer, and a route different from the one printed here is not automatically a worse one. If you reached a different answer and can show your reasoning, bring your script and let us go through it.

Part A — Multiple choice

$5 \times 1 = 5$ marks

A1. (b) the training rows only
The imputer is a *fitted* step: it learns the median from data. Learn it from all rows and the test rows have influenced the value used on them, so the test score is no longer honest.

A2. (c) a decision tree
A tree splits one column at a time at a threshold, and rescaling a column just rescales the thresholds — the same rows fall either side. Distance-based and gradient-based methods, and PCA, all compare magnitudes across columns, so all three care.

A3. (a) the largest variance
PCA finds the direction of greatest variance, then the direction of greatest remaining variance at right angles to it, and so on. It is about spread in the inputs; it never looks at y at all.

A4. (d) η is past the stability threshold $2/L$
A loss that grows every step and overflows is the signature of η past the stability threshold $2/L$: past it each step overshoots by more than the last, so the error grows geometrically from the very first iteration.

A5. (b) unsupervised learning
There is no target column anywhere, so nothing can be supervised. The structure has to come from the inputs themselves.

Part B — Fill in the blanks

$4 \times 1 = 4$ marks

B1. S_{xx}
The slope is $b_1 = S_{xy}/S_{xx}$, where $S_{xx} = \sum(x_i - \bar{x})^2$ measures how spread out the x values are and $S_{xy} = \sum(x_i - \bar{x})(y_i - \bar{y})$ measures how the two move together. Dividing one by the other converts "they move together this much" into "y rises this much per unit of x". It is defined only when $S_{xx} > 0$ — if every x were the same there would be no line to fit.

B2. SSE
SST = SSR + SSE, and this is an *exact* identity rather than an approximation. It holds because of the normal equations: the residuals come out orthogonal to the fitted values, so the cross-term vanishes. SST is the variation you started with, SSR the part the line accounts for, SSE the part it does not — and $R^2 = \text{SSR}/\text{SST} = 1 - \text{SSE}/\text{SST}$ follows straight from it.

B3. 95%
A model that always predicts the majority class is right exactly as often as that class occurs — here 95%. This is why accuracy is close to useless on imbalanced data: it sets a floor that looks impressive and is achieved by a model that has learned nothing and never finds a single positive case. Its recall is 0 and its precision is undefined. Quote precision, recall and F_1 for the rare class instead, and always state what the always-negative baseline would have scored.

B4. reinforcement
Supervised learning has labelled examples; unsupervised has none and must find structure in the inputs alone; *reinforcement* learning has neither, and instead learns a policy from reward collected by acting.

Part C — True or false

$3 \times 1 = 3$ marks

The statement is reproduced first, then the answer and why. A reason was optional on the paper, but writing one is how you find out whether you actually had the idea.

C1. *Putting every fitted step inside a* `Pipeline` *removes every kind of leakage from the analysis.*

False, and the distinction is worth holding onto. A `Pipeline` prevents *preprocessing* leakage — it makes sure every fitted step (scaler, imputer, encoder, selector, resampler) sees training rows only. It does nothing whatever about *target* leakage, which is a feature that would not exist at prediction time or is computed from the answer. No amount of pipeline discipline rescues you from a column that is secretly the target; only understanding what your columns mean does.

C2. R^2 *computed on held-out data cannot be negative.*

False. On held-out data the baseline is the *test* mean, and a model can be worse than simply predicting it — in which case SSE > SST and $R^2 = 1 - \text{SSE/SST}$ goes negative. The $0 \le R^2 \le 1$ range you remember holds for the *training* fit of a model with an intercept, and nowhere else. A negative held-out R^2 is a real and useful signal: it says the model is not merely weak but actively worse than a constant.

C3. *Accuracy cannot be used as the quantity a gradient-based optimiser minimises.*

True. Accuracy counts how many predictions fall on the right side of a threshold, so as a function of the parameters it is a step function — flat almost everywhere, with jumps where a prediction crosses over. Its gradient is zero wherever it exists, so gradient descent has nothing to descend. That is exactly why we train on a smooth surrogate such as cross-entropy and only *report* accuracy.

Part D — Short answer $2 \times 2 = 4$ marks

D1. Fit $\hat{y} = b_0 + b_1 x$ by least squares to the four points $(1, 6)$, $(3, 10)$, $(5, 16)$, $(7, 24)$, using $b_1 = S_{xy}/S_{xx}$ and $b_0 = \bar{y} - b_1 \bar{x}$.

$\bar{x} = 4$ and $\bar{y} = 14$. Building the two sums from the deviations:

$$S_{xx} = \sum (x_i - \bar{x})^2 = \mathbf{20}, \qquad S_{xy} = \sum (x_i - \bar{x})(y_i - \bar{y}) = \mathbf{60}$$

$$b_1 = \frac{S_{xy}}{S_{xx}} = \frac{60}{20} = \mathbf{3}, \qquad b_0 = \bar{y} - b_1 \bar{x} = 14 - (3)(4) = \mathbf{2}$$

So the fitted line is $\hat{y} = 2 + 3x$. Its residuals are $+1$, -1, -1, $+1$, which sum to zero — as they must.

Why $\sum e_i$ had to be zero. You did not need the residuals to know it. Setting $\partial S/\partial b_0 = 0$ gives $-2\sum(y_i - b_0 - b_1 x_i) = 0$, and the bracket *is* the residual — so the first normal equation says exactly $\sum e_i = 0$. The second, $\partial S/\partial b_1 = 0$, says $\sum x_i e_i = 0$. Together they are worth carrying as checks: if your residuals do not sum to zero, the arithmetic is wrong somewhere. They also force the line through $(\bar{x}, \bar{y})$, and make SST = SSR + SSE exact rather than approximate — here $184 = 180 + 4$.

D2. A training loss curve falls quickly, then bounces up and down about a level without ever settling.

(a) Two causes both produce a loss that falls and then bounces without settling.

The first is a **learning rate slightly too large**: the step overshoots the minimum, comes back, overshoots again, and the iterate circles a basin it never lands in. The second is the **gradient-noise floor**: with a small batch and a constant step size, each update is computed from a noisy estimate of the gradient, and once the noise is the same size as the signal the iterate rattles about the optimum instead of converging to it. A third acceptable answer is a schedule that simply needed decaying.

(b) They look identical on the curve, so change one thing and watch. **Lower η (or raise the batch size) and re-run.** If the curve now settles, the step size was the problem. If it still bounces at about the same level, you are at the noise floor — and the fix is different: decay η over time, or average more examples per update. Distinguishing two hypotheses by a single change you can actually make is the whole method here.

Part E — Reasoning

4 marks

E1. The steps below belong to one analysis, listed in no particular order. One of the features, `price_per_sqft`, was computed as Price/Area — and Price is the target.

(1) Standardise the numeric columns. **(2)** Split into training and test sets.
(3) Median-impute the missing areas. **(4)** Drop the `ID` column.
(5) Oversample the minority class. **(6)** Fit the model.

(i) A safe order: 4, 2, 3, 1, 5, 6. Drop `ID` first — it is an identifier and carries no signal. Then **split**. Then, on the training part only, impute the missing areas, standardise, and oversample the minority class. Fit last. Any order works provided the split comes before all three fitted steps and the model is fitted at the end.

(ii) The split. Every step after it learns something *from* the data — a median, a mean and standard deviation, which rows to duplicate. Put any of them before the split and that statistic has been computed partly from the rows you meant to hold back, so the test score is measuring a model that has already seen them. Oversampling before the split is the worst of the three: duplicated minority rows end up in both halves, so the model is tested on rows it trained on.

(iii) `price_per_sqft`. It is computed as price divided by area, and price is the target — so the column hands the model the answer. This is *target* leakage, and unlike the others it does not depend on *where* the step sits. Reorder the pipeline all you like and the column still contains the answer. A `Pipeline` governs what is fitted and when; it has no opinion about what a column means. The test is: *at the moment I make this prediction, would I actually have this value?* Here you would not — you would not know the price.

(iv) Dropping `ID`. It is safe before the split because it fits nothing. Imputing, scaling and oversampling all estimate something from the rows they see; dropping a column is a decision about which columns exist, taken from domain knowledge rather than from the data's values. The same is true of correcting a known data-entry error. Nothing crosses from test to train, so nothing leaks.

Questions on any of this are welcome in the next class or in office hours.

CSAI2017P — Applied Machine Learning (Theory)
Theory Quiz 4 — Solutions • Set B

Maximum marks **20** • 15 questions on four pages

These are the answers to **Set B**, with the reasoning behind each one. Find the set letter on your own paper and work from the matching four pages. Sets were handed out at random and every set covers the same material, so if you want the extra practice the other sets are worth reading too.
Several of these questions have more than one good route to the answer, and a route different from the one printed here is not automatically a worse one. If you reached a different answer and can show your reasoning, bring your script and let us go through it.

Part A — Multiple choice

$5 \times 1 = 5$ marks

A1. (c) the 400 training rows
The split has already happened, so the honest median is the one computed from the 400 training rows. Using all 500 lets the held-back rows influence the value applied to them.

A2. (d) a decision tree
Trees are scale-free because a split is a threshold on one column. Neighbours, gradient descent and PCA all combine columns by magnitude, so an unscaled column in the thousands dominates one in single digits.

A3. (b) 89% of the variance
Explained-variance ratios add: $0.71 + 0.18 = 0.89$. They are shares of the total variance, not counts of features — a component is a direction, not a column.

A4. (a) η is past the stability threshold $2/L$
The losses grow by roughly an order of magnitude each step before overflowing, which is geometric divergence — the signature of a step size past $2/L$.

A5. (c) supervised regression
Sale price is a number on a continuous scale and past prices are available as labels, so this is supervised regression.

Part B — Fill in the blanks

$4 \times 1 = 4$ marks

B1. S_{xx}
The slope is $b_1 = S_{xy}/S_{xx}$, where $S_{xx} = \sum(x_i - \bar{x})^2$ measures how spread out the x values are and $S_{xy} = \sum(x_i - \bar{x})(y_i - \bar{y})$ measures how the two move together. Dividing one by the other converts "they move together this much" into "y rises this much per unit of x". It is defined only when $S_{xx} > 0$ — if every x were the same there would be no line to fit.

B2. SSE
SST = SSR + SSE, and this is an *exact* identity rather than an approximation. It holds because of the normal equations: the residuals come out orthogonal to the fitted values, so the cross-term vanishes. SST is the variation you started with, SSR the part the line accounts for, SSE the part it does not — and $R^2 = \text{SSR}/\text{SST} = 1 - \text{SSE}/\text{SST}$ follows straight from it.

B3. 95%
A model that always predicts the majority class is right exactly as often as that class occurs — here 95%. This is why accuracy is close to useless on imbalanced data: it sets a floor that looks impressive and is achieved by a model that has learned nothing and never finds a single positive case. Its recall is 0 and its precision is undefined. Quote precision, recall and F_1 for the rare class instead, and always state what the always-negative baseline would have scored.

B4. reinforcement
Supervised learning has labelled examples; unsupervised has none and must find structure in the inputs alone; *reinforcement* learning has neither, and instead learns a policy from reward collected by acting.

Part C — True or false

$3 \times 1 = 3$ marks

The statement is reproduced first, then the answer and why. A reason was optional on the paper, but writing one is how you find out whether you actually had the idea.

C1. *A results table produced entirely inside a `Pipeline` is free of every kind of leakage.*

False, and the distinction is worth holding onto. A `Pipeline` prevents *preprocessing* leakage — it makes sure every fitted step (scaler, imputer, encoder, selector, resampler) sees training rows only. It does nothing whatever about *target* leakage, which is a feature that would not exist at prediction time or is computed from the answer. No amount of pipeline discipline rescues you from a column that is secretly the target; only understanding what your columns mean does.

C2. *A table of held-out R^2 values could never legitimately contain a negative entry.*

False. On held-out data the baseline is the *test* mean, and a model can be worse than simply predicting it — in which case SSE $>$ SST and $R^2 = 1 - \text{SSE/SST}$ goes negative. The $0 \leq R^2 \leq 1$ range you remember holds for the *training* fit of a model with an intercept, and nowhere else. A negative held-out R^2 is a real and useful signal: it says the model is not merely weak but actively worse than a constant.

C3. *A column of accuracies cannot serve as the quantity a gradient-based optimiser minimises.*

True. Accuracy counts how many predictions fall on the right side of a threshold, so as a function of the parameters it is a step function — flat almost everywhere, with jumps where a prediction crosses over. Its gradient is zero wherever it exists, so gradient descent has nothing to descend. That is exactly why we train on a smooth surrogate such as cross-entropy and only *report* accuracy.

Part D — Short answer $2 \times 2 = 4$ marks

D1. The table below is to be completed. Fit $\hat{y} = b_0 + b_1 x$ by least squares, using $b_1 = S_{xy}/S_{xx}$ and $b_0 = \bar{y} - b_1\bar{x}$.

x	1	3	5	7	$\sum$
y	8	10	14	20	

$\bar{x} = 4$ and $\bar{y} = 13$. Building the two sums from the deviations:

$$S_{xx} = \sum (x_i - \bar{x})^2 = \mathbf{20}, \qquad S_{xy} = \sum (x_i - \bar{x})(y_i - \bar{y}) = \mathbf{40}$$

$$b_1 = \frac{S_{xy}}{S_{xx}} = \frac{40}{20} = \mathbf{2}, \qquad b_0 = \bar{y} - b_1\bar{x} = 13 - (2)(4) = \mathbf{5}$$

So the fitted line is $\hat{y} = 5 + 2x$. Its residuals are $+1$, -1, -1, $+1$, which sum to zero — as they must.

Why $\sum e_i$ had to be zero. You did not need the residuals to know it. Setting $\partial S/\partial b_0 = 0$ gives $-2\sum(y_i - b_0 - b_1 x_i) = 0$, and the bracket *is* the residual — so the first normal equation says exactly $\sum e_i = 0$. The second, $\partial S/\partial b_1 = 0$, says $\sum x_i e_i = 0$. Together they are worth carrying as checks: if your residuals do not sum to zero, the arithmetic is wrong somewhere. They also force the line through $(\bar{x}, \bar{y})$, and make SST = SSR + SSE exact rather than approximate — here $84 = 80 + 4$.

D2. A table of the training loss at each epoch reads 2.10, 0.88, 0.54, 0.61, 0.49, 0.63, 0.52, 0.60 — it falls, then moves up and down about a level without settling.

(a) Two causes both produce a loss that falls and then bounces without settling.

The first is a **learning rate slightly too large**: the step overshoots the minimum, comes back, overshoots again, and the iterate circles a basin it never lands in. The second is the **gradient-noise floor**: with a small batch and a constant step size, each update is computed from a noisy estimate of the gradient, and once the noise is the same size as the signal the iterate rattles about the optimum instead of converging to it. A third acceptable answer is a schedule that simply needed decaying.

(b) They look identical on the curve, so change one thing and watch. **Lower η (or raise the batch size) and re-run.** If the curve now settles, the step size was the problem. If it still bounces at about the same level, you are at the noise floor — and the fix is different: decay η over time, or average more examples per update. Distinguishing two hypotheses by a single change you can actually make is the whole method here.

Part E — Reasoning

4 marks

E1. A colleague's notebook performs the steps below, listed here in no particular order. One of the features, `price_per_sqft`, was computed as Price/Area, and Price is the target.

(1)	Standardise the numeric columns	(2)	Split into training and test sets
(3)	Median-impute the missing areas	(4)	Drop the `ID` column
(5)	Oversample the minority class	(6)	Fit the model

(i) A safe order: 4, 2, 3, 1, 5, 6. Drop `ID` first — it is an identifier and carries no signal. Then **split**. Then, on the training part only, impute the missing areas, standardise, and oversample the minority class. Fit last. Any order works provided the split comes before all three fitted steps and the model is fitted at the end.

(ii) The split. Every step after it learns something *from* the data — a median, a mean and standard deviation, which rows to duplicate. Put any of them before the split and that statistic has been computed partly from the rows you meant to hold back, so the test score is measuring a model that has already seen them. Oversampling before the split is the worst of the three: duplicated minority rows end up in both halves, so the model is tested on rows it trained on.

(iii) `price_per_sqft`. It is computed as price divided by area, and price is the target — so the column hands the model the answer. This is *target* leakage, and unlike the others it does not depend on *where* the step sits. Reorder the pipeline all you like and the column still contains the answer. A `Pipeline` governs what is fitted and when; it has no opinion about what a column means. The test is: *at the moment I make this prediction, would I actually have this value?* Here you would not — you would not know the price.

(iv) Dropping `ID`. It is safe before the split because it fits nothing. Imputing, scaling and oversampling all estimate something from the rows they see; dropping a column is a decision about which columns exist, taken from domain knowledge rather than from the data's values. The same is true of correcting a known data-entry error. Nothing crosses from test to train, so nothing leaks.

Questions on any of this are welcome in the next class or in office hours.

CSAI2017P — Applied Machine Learning (Theory)
Theory Quiz 4 — Solutions • Set C

Maximum marks **20** • 15 questions on four pages

These are the answers to **Set C**, with the reasoning behind each one. Find the set letter on your own paper and work from the matching four pages. Sets were handed out at random and every set covers the same material, so if you want the extra practice the other sets are worth reading too.
Several of these questions have more than one good route to the answer, and a route different from the one printed here is not automatically a worse one. If you reached a different answer and can show your reasoning, bring your script and let us go through it.

Part A — Multiple choice

$5 \times 1 = 5$ marks

A1. (d) the training rows only
A skewed column is exactly where the median earns its keep over the mean — but either way it is fitted from data, so it must come from the training rows only.

A2. (a) a decision tree
One axis spanning 0–40000 against another spanning 0–3 is precisely the situation that breaks distance-based methods. A tree does not measure distance, so the picture makes no difference to it.

A3. (c) how many components to keep
The elbow is where extra components stop buying much variance, which is how you choose how many to keep. It says nothing about the target — PCA never sees y.

A4. (b) η is past the stability threshold $2/L$
A curve that leaves the top of the axis and then stops being plotted is a loss that has overflowed: the step size is past the stability threshold.

A5. (d) unsupervised learning
No colour coding means no labels, so the clumps have to come from the points themselves.

Part B — Fill in the blanks

$4 \times 1 = 4$ marks

B1. S_{xx}
The slope is $b_1 = S_{xy}/S_{xx}$, where $S_{xx} = \sum(x_i - \bar{x})^2$ measures how spread out the x values are and $S_{xy} = \sum(x_i - \bar{x})(y_i - \bar{y})$ measures how the two move together. Dividing one by the other converts "they move together this much" into "y rises this much per unit of x". It is defined only when $S_{xx} > 0$ — if every x were the same there would be no line to fit.

B2. SSE
SST = SSR + SSE, and this is an *exact* identity rather than an approximation. It holds because of the normal equations: the residuals come out orthogonal to the fitted values, so the cross-term vanishes. SST is the variation you started with, SSR the part the line accounts for, SSE the part it does not — and $R^2 = \text{SSR}/\text{SST} = 1 - \text{SSE}/\text{SST}$ follows straight from it.

B3. 95%
A model that always predicts the majority class is right exactly as often as that class occurs — here 95%. This is why accuracy is close to useless on imbalanced data: it sets a floor that looks impressive and is achieved by a model that has learned nothing and never finds a single positive case. Its recall is 0 and its precision is undefined. Quote precision, recall and F_1 for the rare class instead, and always state what the always-negative baseline would have scored.

B4. reinforcement
Supervised learning has labelled examples; unsupervised has none and must find structure in the inputs alone; *reinforcement* learning has neither, and instead learns a policy from reward collected by acting.

Part C — True or false

$3 \times 1 = 3$ marks

The statement is reproduced first, then the answer and why. A reason was optional on the paper, but writing one is how you find out whether you actually had the idea.

C1. *A pipeline diagram in which every fitted box sits after the split is, by that fact alone, free of all leakage.*

False, and the distinction is worth holding onto. A `Pipeline` prevents *preprocessing* leakage — it makes sure every fitted step (scaler, imputer, encoder, selector, resampler) sees training rows only. It does nothing whatever about *target* leakage, which is a feature that would not exist at prediction time or is computed from the answer. No amount of pipeline discipline rescues you from a column that is secretly the target; only understanding what your columns mean does.

C2. *A plot of held-out R^2 against model complexity can never dip below zero.*

False. On held-out data the baseline is the *test* mean, and a model can be worse than simply predicting it — in which case SSE $>$ SST and $R^2 = 1 - \text{SSE}/\text{SST}$ goes negative. The $0 \le R^2 \le 1$ range you remember holds for the *training* fit of a model with an intercept, and nowhere else. A negative held-out R^2 is a real and useful signal: it says the model is not merely weak but actively worse than a constant.

C3. *A plot of accuracy against a model parameter is a step function, which is why it cannot be the quantity minimised by gradient descent.*

True. Accuracy counts how many predictions fall on the right side of a threshold, so as a function of the parameters it is a step function — flat almost everywhere, with jumps where a prediction crosses over. Its gradient is zero wherever it exists, so gradient descent has nothing to descend. That is exactly why we train on a smooth surrogate such as cross-entropy and only *report* accuracy.

Part D — Short answer $2 \times 2 = 4$ marks

D1. Four points are plotted: $(1, 6)$, $(3, 12)$, $(5, 20)$, $(7, 30)$. A straight line is to be drawn through them by least squares, using $b_1 = S_{xy}/S_{xx}$ and $b_0 = \bar{y} - b_1\bar{x}$.

$\bar{x} = 4$ and $\bar{y} = 17$. Building the two sums from the deviations:

$$S_{xx} = \sum (x_i - \bar{x})^2 = \mathbf{20}, \qquad S_{xy} = \sum (x_i - \bar{x})(y_i - \bar{y}) = \mathbf{80}$$

$$b_1 = \frac{S_{xy}}{S_{xx}} = \frac{80}{20} = \mathbf{4}, \qquad b_0 = \bar{y} - b_1\bar{x} = 17 - (4)(4) = \mathbf{1}$$

So the fitted line is $\hat{y} = 1 + 4x$. Its residuals are $+1$, -1, -1, $+1$, which sum to zero — as they must.

Why $\sum e_i$ had to be zero. You did not need the residuals to know it. Setting $\partial S/\partial b_0 = 0$ gives $-2\sum(y_i - b_0 - b_1 x_i) = 0$, and the bracket *is* the residual — so the first normal equation says exactly $\sum e_i = 0$. The second, $\partial S/\partial b_1 = 0$, says $\sum x_i e_i = 0$. Together they are worth carrying as checks: if your residuals do not sum to zero, the arithmetic is wrong somewhere. They also force the line through $(\bar{x}, \bar{y})$, and make SST = SSR + SSE exact rather than approximate — here $324 = 320 + 4$.

D2. A plotted loss curve drops steeply for the first few epochs and then oscillates about a horizontal level, never settling onto it.

(a) Two causes both produce a loss that falls and then bounces without settling.

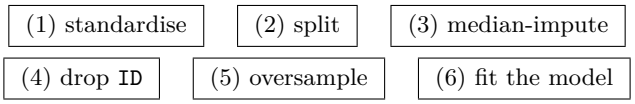
The first is a **learning rate slightly too large**: the step overshoots the minimum, comes back, overshoots again, and the iterate circles a basin it never lands in. The second is the **gradient-noise floor**: with a small batch and a constant step size, each update is computed from a noisy estimate of the gradient, and once the noise is the same size as the signal the iterate rattles about the optimum instead of converging to it. A third acceptable answer is a schedule that simply needed decaying.

(b) They look identical on the curve, so change one thing and watch. **Lower η (or raise the batch size) and re-run.** If the curve now settles, the step size was the problem. If it still bounces at about the same level, you are at the noise floor — and the fix is different: decay η over time, or average more examples per update. Distinguishing two hypotheses by a single change you can actually make is the whole method here.

Part E — Reasoning

4 marks

E1. A pipeline diagram has been drawn with its boxes in no particular order. One of the features, `price_per_sqft`, was computed as Price/Area, and Price is the target.



(i) A safe order: 4, 2, 3, 1, 5, 6. Drop `ID` first — it is an identifier and carries no signal. Then **split**. Then, on the training part only, impute the missing areas, standardise, and oversample the minority class. Fit last. Any order works provided the split comes before all three fitted steps and the model is fitted at the end.

(ii) The split. Every step after it learns something *from* the data — a median, a mean and standard deviation, which rows to duplicate. Put any of them before the split and that statistic has been computed partly from the rows you meant to hold back, so the test score is measuring a model that has already seen them. Oversampling before the split is the worst of the three: duplicated minority rows end up in both halves, so the model is tested on rows it trained on.

(iii) `price_per_sqft`. It is computed as price divided by area, and price is the target — so the column hands the model the answer. This is *target* leakage, and unlike the others it does not depend on *where* the step sits. Reorder the pipeline all you like and the column still contains the answer. A `Pipeline` governs what is fitted and when; it has no opinion about what a column means. The test is: *at the moment I make this prediction, would I actually have this value?* Here you would not — you would not know the price.

(iv) Dropping `ID`. It is safe before the split because it fits nothing. Imputing, scaling and oversampling all estimate something from the rows they see; dropping a column is a decision about which columns exist, taken from domain knowledge rather than from the data's values. The same is true of correcting a known data-entry error. Nothing crosses from test to train, so nothing leaks.

Questions on any of this are welcome in the next class or in office hours.

CSAI2017P — Applied Machine Learning (Theory)
Theory Quiz 4 — Solutions • Set D

Maximum marks **20** • 15 questions on four pages

These are the answers to **Set D**, with the reasoning behind each one. Find the set letter on your own paper and work from the matching four pages. Sets were handed out at random and every set covers the same material, so if you want the extra practice the other sets are worth reading too.
Several of these questions have more than one good route to the answer, and a route different from the one printed here is not automatically a worse one. If you reached a different answer and can show your reasoning, bring your script and let us go through it.

Part A — Multiple choice

$5 \times 1 = 5$ marks

A1. (a) the training rows only
The median is learned from data, so learning it from every row lets the held-back patients influence the values used to score the model on them.

A2. (b) a decision tree
A tree asks "is income above this threshold?", and rescaling income just rescales the threshold. Neighbours, gradient descent and PCA all compare magnitudes across columns and would let lakhs swamp a count of dependants.

A3. (d) principal component analysis
Forty correlated columns replaced by a handful that retain most of the variation is the definition of PCA. One-hot encoding expands columns; oversampling adds rows; a split divides rows.

A4. (c) η is past the stability threshold $2/L$
Exploding within a few iterations, rather than after many, points at the step size rather than anything about the data or the model's size.

A5. (a) unsupervised learning
No customer has ever been labelled, so there is nothing to learn a mapping to.

Part B — Fill in the blanks

$4 \times 1 = 4$ marks

B1. S_{xx}
The slope is $b_1 = S_{xy}/S_{xx}$, where $S_{xx} = \sum(x_i - \bar{x})^2$ measures how spread out the x values are and $S_{xy} = \sum(x_i - \bar{x})(y_i - \bar{y})$ measures how the two move together. Dividing one by the other converts "they move together this much" into "y rises this much per unit of x". It is defined only when $S_{xx} > 0$ — if every x were the same there would be no line to fit.

B2. SSE
SST = SSR + SSE, and this is an *exact* identity rather than an approximation. It holds because of the normal equations: the residuals come out orthogonal to the fitted values, so the cross-term vanishes. SST is the variation you started with, SSR the part the line accounts for, SSE the part it does not — and $R^2 = \text{SSR}/\text{SST} = 1 - \text{SSE}/\text{SST}$ follows straight from it.

B3. 95%
A model that always predicts the majority class is right exactly as often as that class occurs — here 95%. This is why accuracy is close to useless on imbalanced data: it sets a floor that looks impressive and is achieved by a model that has learned nothing and never finds a single positive case. Its recall is 0 and its precision is undefined. Quote precision, recall and F_1 for the rare class instead, and always state what the always-negative baseline would have scored.

B4. reinforcement
Supervised learning has labelled examples; unsupervised has none and must find structure in the inputs alone; *reinforcement* learning has neither, and instead learns a policy from reward collected by acting.

Part C — True or false

$3 \times 1 = 3$ marks

The statement is reproduced first, then the answer and why. A reason was optional on the paper, but writing one is how you find out whether you actually had the idea.

C1. *A consultant who has put every fitted step inside a* `Pipeline` *can assure the client there is no leakage of any kind.*

False, and the distinction is worth holding onto. A `Pipeline` prevents *preprocessing* leakage — it makes sure every fitted step (scaler, imputer, encoder, selector, resampler) sees training rows only. It does nothing whatever about *target* leakage, which is a feature that would not exist at prediction time or is computed from the answer. No amount of pipeline discipline rescues you from a column that is secretly the target; only understanding what your columns mean does.

C2. *A team reporting R^2 on a held-out set can be sure the figure will not come out negative.*

False. On held-out data the baseline is the *test* mean, and a model can be worse than simply predicting it — in which case SSE > SST and $R^2 = 1 - \text{SSE}/\text{SST}$ goes negative. The $0 \le R^2 \le 1$ range you remember holds for the *training* fit of a model with an intercept, and nowhere else. A negative held-out R^2 is a real and useful signal: it says the model is not merely weak but actively worse than a constant.

C3. *An engineer cannot train a model by asking gradient descent to maximise accuracy directly.*

True. Accuracy counts how many predictions fall on the right side of a threshold, so as a function of the parameters it is a step function — flat almost everywhere, with jumps where a prediction crosses over. Its gradient is zero wherever it exists, so gradient descent has nothing to descend. That is exactly why we train on a smooth surrogate such as cross-entropy and only *report* accuracy.

Part D — Short answer $2 \times 2 = 4$ marks

D1. A workshop records the number of faults found (y) against the number of hours spent inspecting (x) on four days: $(1, 12)$, $(3, 12)$, $(5, 14)$, $(7, 18)$. Fit $\hat{y} = b_0 + b_1 x$ by least squares, using $b_1 = S_{xy}/S_{xx}$ and $b_0 = \bar{y} - b_1\bar{x}$.

$\bar{x} = 4$ and $\bar{y} = 14$. Building the two sums from the deviations:

$$S_{xx} = \sum (x_i - \bar{x})^2 = \mathbf{20}, \qquad S_{xy} = \sum (x_i - \bar{x})(y_i - \bar{y}) = \mathbf{20}$$

$$b_1 = \frac{S_{xy}}{S_{xx}} = \frac{20}{20} = \mathbf{1}, \qquad b_0 = \bar{y} - b_1\bar{x} = 14 - (1)(4) = \mathbf{10}$$

So the fitted line is $\hat{y} = 10 + 1x$. Its residuals are $+1$, -1, -1, $+1$, which sum to zero — as they must.

Why $\sum e_i$ had to be zero. You did not need the residuals to know it. Setting $\partial S/\partial b_0 = 0$ gives $-2\sum(y_i - b_0 - b_1 x_i) = 0$, and the bracket *is* the residual — so the first normal equation says exactly $\sum e_i = 0$. The second, $\partial S/\partial b_1 = 0$, says $\sum x_i e_i = 0$. Together they are worth carrying as checks: if your residuals do not sum to zero, the arithmetic is wrong somewhere. They also force the line through $(\bar{x}, \bar{y})$, and make SST = SSR + SSE exact rather than approximate — here $24 = 20 + 4$.

D2. An engineer reports that her training loss drops nicely for a few epochs and then simply bounces around a level, epoch after epoch, without ever settling.

(a) Two causes both produce a loss that falls and then bounces without settling.

The first is a **learning rate slightly too large**: the step overshoots the minimum, comes back, overshoots again, and the iterate circles a basin it never lands in. The second is the **gradient-noise floor**: with a small batch and a constant step size, each update is computed from a noisy estimate of the gradient, and once the noise is the same size as the signal the iterate rattles about the optimum instead of converging to it. A third acceptable answer is a schedule that simply needed decaying.

(b) They look identical on the curve, so change one thing and watch. **Lower η (or raise the batch size) and re-run.** If the curve now settles, the step size was the problem. If it still bounces at about the same level, you are at the noise floor — and the fix is different: decay η over time, or average more examples per update. Distinguishing two hypotheses by a single change you can actually make is the whole method here.

Part E — Reasoning

4 marks

E1. An intern hands you a notebook. It does the six things below, though not in this order, and one of the features — `price_per_sqft` — was computed as Price/Area, where Price is the very thing the model is meant to predict.

(1) Standardise the numeric columns. **(2)** Split into training and test sets.
(3) Median-impute the missing areas. **(4)** Drop the `ID` column.
(5) Oversample the minority class. **(6)** Fit the model.

(i) A safe order: 4, 2, 3, 1, 5, 6. Drop `ID` first — it is an identifier and carries no signal. Then **split**. Then, on the training part only, impute the missing areas, standardise, and oversample the minority class. Fit last. Any order works provided the split comes before all three fitted steps and the model is fitted at the end.

(ii) The split. Every step after it learns something *from* the data — a median, a mean and standard deviation, which rows to duplicate. Put any of them before the split and that statistic has been computed partly from the rows you meant to hold back, so the test score is measuring a model that has already seen them. Oversampling before the split is the worst of the three: duplicated minority rows end up in both halves, so the model is tested on rows it trained on.

(iii) `price_per_sqft`. It is computed as price divided by area, and price is the target — so the column hands the model the answer. This is *target* leakage, and unlike the others it does not depend on *where* the step sits. Reorder the pipeline all you like and the column still contains the answer. A `Pipeline` governs what is fitted and when; it has no opinion about what a column means. The test is: *at the moment I make this prediction, would I actually have this value?* Here you would not — you would not know the price.

(iv) Dropping `ID`. It is safe before the split because it fits nothing. Imputing, scaling and oversampling all estimate something from the rows they see; dropping a column is a decision about which columns exist, taken from domain knowledge rather than from the data's values. The same is true of correcting a known data-entry error. Nothing crosses from test to train, so nothing leaks.

Questions on any of this are welcome in the next class or in office hours.

CSAI2017P — Applied Machine Learning (Theory)
Theory Quiz 4 — Solutions • Set E

Maximum marks **20** • 15 questions on four pages

These are the answers to **Set E**, with the reasoning behind each one. Find the set letter on your own paper and work from the matching four pages. Sets were handed out at random and every set covers the same material, so if you want the extra practice the other sets are worth reading too.
Several of these questions have more than one good route to the answer, and a route different from the one printed here is not automatically a worse one. If you reached a different answer and can show your reasoning, bring your script and let us go through it.

Part A — Multiple choice

$5 \times 1 = 5$ marks

A1. (b) the training rows only
Inside a `Pipeline`, `fit` runs on the training fold and `transform` is then applied to the test fold using what was learned from training. That is the whole point of putting it in the pipeline.

A2. (d) `DecisionTreeRegressor`
`DecisionTreeRegressor` splits one feature at a time at a threshold, so rescaling changes nothing. The other three all compare magnitudes across features.

A3. (a) 89% of the variance
`explained_variance_ratio_` entries are shares of the total variance and simply add: $0.71 + 0.18 = 0.89$.

A4. (c) `eta` is past $2/L$
Each printed loss is roughly an order of magnitude above the last before overflowing — geometric divergence, which means `eta` is past $2/L$.

A5. (d) unsupervised learning
No `y` exists anywhere, so nothing is supervised; `KMeans` finds structure in `X` alone.

Part B — Fill in the blanks

$4 \times 1 = 4$ marks

B1. S_{xx}
The slope is $b_1 = S_{xy}/S_{xx}$, where $S_{xx} = \sum(x_i - \bar{x})^2$ measures how spread out the x values are and $S_{xy} = \sum(x_i - \bar{x})(y_i - \bar{y})$ measures how the two move together. Dividing one by the other converts "they move together this much" into "y rises this much per unit of x". It is defined only when $S_{xx} > 0$ — if every x were the same there would be no line to fit.

B2. SSE
SST = SSR + SSE, and this is an *exact* identity rather than an approximation. It holds because of the normal equations: the residuals come out orthogonal to the fitted values, so the cross-term vanishes. SST is the variation you started with, SSR the part the line accounts for, SSE the part it does not — and $R^2 = \text{SSR}/\text{SST} = 1 - \text{SSE}/\text{SST}$ follows straight from it.

B3. 95%
A model that always predicts the majority class is right exactly as often as that class occurs — here 95%. This is why accuracy is close to useless on imbalanced data: it sets a floor that looks impressive and is achieved by a model that has learned nothing and never finds a single positive case. Its recall is 0 and its precision is undefined. Quote precision, recall and F_1 for the rare class instead, and always state what the always-negative baseline would have scored.

B4. reinforcement
Supervised learning has labelled examples; unsupervised has none and must find structure in the inputs alone; *reinforcement* learning has neither, and instead learns a policy from reward collected by acting.

Part C — True or false

$3 \times 1 = 3$ marks

The statement is reproduced first, then the answer and why. A reason was optional on the paper, but writing one is how you find out whether you actually had the idea.

C1. *Wrapping every step in `make_pipeline(...)` guarantees that no leakage of any kind remains.*

False, and the distinction is worth holding onto. A `Pipeline` prevents *preprocessing* leakage — it makes sure every fitted step (scaler, imputer, encoder, selector, resampler) sees training rows only. It does nothing whatever about *target* leakage, which is a feature that would not exist at prediction time or is computed from the answer. No amount of pipeline discipline rescues you from a column that is secretly the target; only understanding what your columns mean does.

C2. *`r2_score(y_test, pred)` can never return a negative number.*

False. On held-out data the baseline is the *test* mean, and a model can be worse than simply predicting it — in which case SSE > SST and $R^2 = 1 - \text{SSE}/\text{SST}$ goes negative. The $0 \le R^2 \le 1$ range you remember holds for the *training* fit of a model with an intercept, and nowhere else. A negative held-out R^2 is a real and useful signal: it says the model is not merely weak but actively worse than a constant.

C3. *`accuracy_score` cannot be handed to a gradient-based optimiser as the objective to minimise.*

True. Accuracy counts how many predictions fall on the right side of a threshold, so as a function of the parameters it is a step function — flat almost everywhere, with jumps where a prediction crosses over. Its gradient is zero wherever it exists, so gradient descent has nothing to descend. That is exactly why we train on a smooth surrogate such as cross-entropy and only *report* accuracy.

Part D — Short answer

$2 \times 2 = 4$ marks

D1. `x = np.array([1, 3, 5, 7])` and `y = np.array([6, 14, 24, 36])`. Fit $\hat{y} = b_0 + b_1 x$ by hand, using $b_1 = S_{xy}/S_{xx}$ and $b_0 = \bar{y} - b_1\bar{x}$ — that is, what this would print:

```
b1 = ((x - x.mean()) * (y - y.mean())).sum() / ((x - x.mean())**2).sum()
b0 = y.mean() - b1 * x.mean()
print(b1, b0)
```

$\bar{x} = 4$ and $\bar{y} = 20$. Building the two sums from the deviations:

$$S_{xx} = \sum (x_i - \bar{x})^2 = \mathbf{20}, \qquad S_{xy} = \sum (x_i - \bar{x})(y_i - \bar{y}) = \mathbf{100}$$

$$b_1 = \frac{S_{xy}}{S_{xx}} = \frac{100}{20} = \mathbf{5}, \qquad b_0 = \bar{y} - b_1\bar{x} = 20 - (5)(4) = \mathbf{0}$$

So the fitted line is $\hat{y} = 0 + 5x$. Its residuals are $+1$, -1, -1, $+1$, which sum to zero — as they must.

Why $\sum e_i$ had to be zero. You did not need the residuals to know it. Setting $\partial S/\partial b_0 = 0$ gives $-2\sum(y_i - b_0 - b_1 x_i) = 0$, and the bracket *is* the residual — so the first normal equation says exactly $\sum e_i = 0$. The second, $\partial S/\partial b_1 = 0$, says $\sum x_i e_i = 0$. Together they are worth carrying as checks: if your residuals do not sum to zero, the arithmetic is wrong somewhere. They also force the line through $(\bar{x}, \bar{y})$, and make SST = SSR + SSE exact rather than approximate — here $504 = 500 + 4$.

D2. A training loop prints the loss each epoch: it falls steeply, then prints values that move up and down about one level and stay there.

(a) Two causes both produce a loss that falls and then bounces without settling.

The first is a **learning rate slightly too large**: the step overshoots the minimum, comes back, overshoots again, and the iterate circles a basin it never lands in. The second is the **gradient-noise floor**: with a small batch and a constant step size, each update is computed from a noisy estimate of the gradient, and once the noise is the same size as the signal the iterate rattles about the optimum instead of converging to it. A third acceptable answer is a schedule that simply needed decaying.

(b) They look identical on the curve, so change one thing and watch. **Lower η (or raise the batch size) and re-run.** If the curve now settles, the step size was the problem. If it still bounces at about the same level, you are at the noise floor — and the fix is different: decay η over time, or average more examples per update. Distinguishing two hypotheses by a single change you can actually make is the whole method here.

Part E — Reasoning

4 marks

E1. The cell below runs without error. `price_per_sqft` is computed from `Price_lakh`, which is the target.

```
df["price_per_sqft"] = df["Price_lakh"] / df["Area_sqft"]
df = df.fillna(df.median(numeric_only=True))
X = StandardScaler().fit_transform(df.drop(columns=["Price_lakh"]))
X_tr, X_te, y_tr, y_te = train_test_split(X, df["Price_lakh"])
```

(i) A safe order: 4, 2, 3, 1, 5, 6. Drop `ID` first — it is an identifier and carries no signal. Then **split**. Then, on the training part only, impute the missing areas, standardise, and oversample the minority class. Fit last. Any order works provided the split comes before all three fitted steps and the model is fitted at the end.

(ii) The split. Every step after it learns something *from* the data — a median, a mean and standard deviation, which rows to duplicate. Put any of them before the split and that statistic has been computed partly from the rows you meant to hold back, so the test score is measuring a model that has already seen them. Oversampling before the split is the worst of the three: duplicated minority rows end up in both halves, so the model is tested on rows it trained on.

(iii) `price_per_sqft`. It is computed as price divided by area, and price is the target — so the column hands the model the answer. This is *target* leakage, and unlike the others it does not depend on *where* the step sits. Reorder the pipeline all you like and the column still contains the answer. A `Pipeline` governs what is fitted and when; it has no opinion about what a column means. The test is: *at the moment I make this prediction, would I actually have this value?* Here you would not — you would not know the price.

(iv) Dropping `ID`. It is safe before the split because it fits nothing. Imputing, scaling and oversampling all estimate something from the rows they see; dropping a column is a decision about which columns exist, taken from domain knowledge rather than from the data's values. The same is true of correcting a known data-entry error. Nothing crosses from test to train, so nothing leaks.

Questions on any of this are welcome in the next class or in office hours.

CSAI2017P — Applied Machine Learning (Theory)
Theory Quiz 4 — Solutions • Set F

Maximum marks **20** • 15 questions on four pages

These are the answers to **Set F**, with the reasoning behind each one. Find the set letter on your own paper and work from the matching four pages. Sets were handed out at random and every set covers the same material, so if you want the extra practice the other sets are worth reading too.
Several of these questions have more than one good route to the answer, and a route different from the one printed here is not automatically a worse one. If you reached a different answer and can show your reasoning, bring your script and let us go through it.

Part A — Multiple choice

$5 \times 1 = 5$ marks

A1. (c) the training rows alone
The middle value is worked out *from* the data, so working it out from every row lets the held-back rows influence the number applied to them.

A2. (a) a decision tree
A method that asks only "is this column above that cut?" does not care what units the column is in. Methods that add or compare columns do.

A3. (b) principal component analysis
Replacing many overlapping columns with a few that keep most of the variation is what principal components are for.

A4. (d) the step size is past the stability limit
A loss that grows every step and then stops being a number has overflowed, and that is what a step size past the stability limit does.

A5. (a) unsupervised learning
Nobody has ever labelled a customer, so there is no target to learn.

Part B — Fill in the blanks

$4 \times 1 = 4$ marks

B1. S_{xx}
The slope is $b_1 = S_{xy}/S_{xx}$, where $S_{xx} = \sum(x_i - \bar{x})^2$ measures how spread out the x values are and $S_{xy} = \sum(x_i - \bar{x})(y_i - \bar{y})$ measures how the two move together. Dividing one by the other converts "they move together this much" into "y rises this much per unit of x". It is defined only when $S_{xx} > 0$ — if every x were the same there would be no line to fit.

B2. SSE
SST = SSR + SSE, and this is an *exact* identity rather than an approximation. It holds because of the normal equations: the residuals come out orthogonal to the fitted values, so the cross-term vanishes. SST is the variation you started with, SSR the part the line accounts for, SSE the part it does not — and $R^2 = \text{SSR}/\text{SST} = 1 - \text{SSE}/\text{SST}$ follows straight from it.

B3. 95%
A model that always predicts the majority class is right exactly as often as that class occurs — here 95%. This is why accuracy is close to useless on imbalanced data: it sets a floor that looks impressive and is achieved by a model that has learned nothing and never finds a single positive case. Its recall is 0 and its precision is undefined. Quote precision, recall and F_1 for the rare class instead, and always state what the always-negative baseline would have scored.

B4. reinforcement
Supervised learning has labelled examples; unsupervised has none and must find structure in the inputs alone; *reinforcement* learning has neither, and instead learns a policy from reward collected by acting.

Part C — True or false

$3 \times 1 = 3$ marks

The statement is reproduced first, then the answer and why. A reason was optional on the paper, but writing one is how you find out whether you actually had the idea.

C1. *Once every fitted step has been placed inside a pipeline, no kind of leakage can remain.*

False, and the distinction is worth holding onto. A `Pipeline` prevents *preprocessing* leakage — it makes sure every fitted step (scaler, imputer, encoder, selector, resampler) sees training rows only. It does nothing whatever about *target* leakage, which is a feature that would not exist at prediction time or is computed from the answer. No amount of pipeline discipline rescues you from a column that is secretly the target; only understanding what your columns mean does.

C2. *The share of variation explained, measured on data the model has never seen, can never come out below zero.*

False. On held-out data the baseline is the *test* mean, and a model can be worse than simply predicting it — in which case SSE $>$ SST and $R^2 = 1 - \text{SSE}/\text{SST}$ goes negative. The $0 \leq R^2 \leq 1$ range you remember holds for the *training* fit of a model with an intercept, and nowhere else. A negative held-out R^2 is a real and useful signal: it says the model is not merely weak but actively worse than a constant.

C3. *The proportion of cases got right cannot itself be the quantity a gradient method is asked to improve.*

True. Accuracy counts how many predictions fall on the right side of a threshold, so as a function of the parameters it is a step function — flat almost everywhere, with jumps where a prediction crosses over. Its gradient is zero wherever it exists, so gradient descent has nothing to descend. That is exactly why we train on a smooth surrogate such as cross-entropy and only *report* accuracy.

Part D — Short answer

$2 \times 2 = 4$ marks

D1. Four readings were taken. At $x = 1, 3, 5, 7$ the measured values were 7, 1, -3 and -5. Fit a straight line by least squares: the slope is the cross-product sum divided by the sum of squared deviations in x, and the intercept then follows from the line passing through the point of means.

$\bar{x} = 4$ and $\bar{y} = 0$. Building the two sums from the deviations:

$$S_{xx} = \sum (x_i - \bar{x})^2 = \mathbf{20}, \qquad S_{xy} = \sum (x_i - \bar{x})(y_i - \bar{y}) = \mathbf{\text{-}40}$$

$$b_1 = \frac{S_{xy}}{S_{xx}} = \frac{-40}{20} = \mathbf{\text{-}2}, \qquad b_0 = \bar{y} - b_1 \bar{x} = 0 - (-2)(4) = \mathbf{8}$$

So the fitted line is $\hat{y} = 8 - 2x$. Its residuals are $+1$, -1, -1, $+1$, which sum to zero — as they must.

Why $\sum e_i$ had to be zero. You did not need the residuals to know it. Setting $\partial S / \partial b_0 = 0$ gives $-2\sum(y_i - b_0 - b_1 x_i) = 0$, and the bracket *is* the residual — so the first normal equation says exactly $\sum e_i = 0$. The second, $\partial S / \partial b_1 = 0$, says $\sum x_i e_i = 0$. Together they are worth carrying as checks: if your residuals do not sum to zero, the arithmetic is wrong somewhere. They also force the line through $(\bar{x}, \bar{y})$, and make SST = SSR + SSE exact rather than approximate — here $84 = 80 + 4$.

D2. A training run's loss falls sharply at first and then, epoch after epoch, simply moves up and down about one level without ever settling on it.

(a) Two causes both produce a loss that falls and then bounces without settling.

The first is a **learning rate slightly too large**: the step overshoots the minimum, comes back, overshoots again, and the iterate circles a basin it never lands in. The second is the **gradient-noise floor**: with a small batch and a constant step size, each update is computed from a noisy estimate of the gradient, and once the noise is the same size as the signal the iterate rattles about the optimum instead of converging to it. A third acceptable answer is a schedule that simply needed decaying.

(b) They look identical on the curve, so change one thing and watch. **Lower η (or raise the batch size) and re-run.** If the curve now settles, the step size was the problem. If it still bounces at about the same level, you are at the noise floor — and the fix is different: decay η over time, or average more examples per update. Distinguishing two hypotheses by a single change you can actually make is the whole method here.

Part E — Reasoning

4 marks

E1. A colleague describes what her notebook does, though not in the order she does it. One of the columns she feeds the model is the price divided by the area — and the price is the very thing she is trying to predict.

(1) Put every numeric column on a common scale. **(2)** Set part of the data aside for testing.
(3) Fill the missing areas with the middle value. **(4)** Throw away the identifier column.
(5) Duplicate rows of the rarer class. **(6)** Fit the model.

(i) A safe order: 4, 2, 3, 1, 5, 6. Drop `ID` first — it is an identifier and carries no signal. Then **split**. Then, on the training part only, impute the missing areas, standardise, and oversample the minority class. Fit last. Any order works provided the split comes before all three fitted steps and the model is fitted at the end.

(ii) The split. Every step after it learns something *from* the data — a median, a mean and standard deviation, which rows to duplicate. Put any of them before the split and that statistic has been computed partly from the rows you meant to hold back, so the test score is measuring a model that has already seen them. Oversampling before the split is the worst of the three: duplicated minority rows end up in both halves, so the model is tested on rows it trained on.

(iii) `price_per_sqft`. It is computed as price divided by area, and price is the target — so the column hands the model the answer. This is *target* leakage, and unlike the others it does not depend on *where* the step sits. Reorder the pipeline all you like and the column still contains the answer. A `Pipeline` governs what is fitted and when; it has no opinion about what a column means. The test is: *at the moment I make this prediction, would I actually have this value?* Here you would not — you would not know the price.

(iv) Dropping `ID`. It is safe before the split because it fits nothing. Imputing, scaling and oversampling all estimate something from the rows they see; dropping a column is a decision about which columns exist, taken from domain knowledge rather than from the data's values. The same is true of correcting a known data-entry error. Nothing crosses from test to train, so nothing leaks.

Questions on any of this are welcome in the next class or in office hours.

CSAI2017P — Applied Machine Learning (Theory)
Theory Quiz 4 — Solutions • Set G

Maximum marks **20** • 15 questions on four pages

These are the answers to **Set G**, with the reasoning behind each one. Find the set letter on your own paper and work from the matching four pages. Sets were handed out at random and every set covers the same material, so if you want the extra practice the other sets are worth reading too.
Several of these questions have more than one good route to the answer, and a route different from the one printed here is not automatically a worse one. If you reached a different answer and can show your reasoning, bring your script and let us go through it.

Part A — Multiple choice

$5 \times 1 = 5$ marks

A1. (d) only the first gives an honest test score
Only the training median keeps the test score honest. The all-rows median has been computed partly from the rows you are about to score on.

A2. (b) the tree is unaffected, the neighbours are not
A tree's splits are thresholds on single columns, so rescaling moves the thresholds and nothing else. Neighbours measure distance, which mixes the columns together.

A3. (c) the first carries more variance
By construction the first component takes the largest variance available, and the second the largest of what is left at right angles to it.

A4. (a) η is past $2/L$
A loss that flattens has converged. A loss that rises to overflow has a step size past $2/L$ — the two curves are not two degrees of the same problem.

A5. (b) (i) unsupervised, (ii) supervised regression
Groups with no labels is unsupervised; a continuous labelled target is supervised regression.

Part B — Fill in the blanks

$4 \times 1 = 4$ marks

B1. S_{xx}
The slope is $b_1 = S_{xy}/S_{xx}$, where $S_{xx} = \sum(x_i - \bar{x})^2$ measures how spread out the x values are and $S_{xy} = \sum(x_i - \bar{x})(y_i - \bar{y})$ measures how the two move together. Dividing one by the other converts "they move together this much" into "y rises this much per unit of x". It is defined only when $S_{xx} > 0$ — if every x were the same there would be no line to fit.

B2. SSE
SST = SSR + SSE, and this is an *exact* identity rather than an approximation. It holds because of the normal equations: the residuals come out orthogonal to the fitted values, so the cross-term vanishes. SST is the variation you started with, SSR the part the line accounts for, SSE the part it does not — and $R^2 = \text{SSR}/\text{SST} = 1 - \text{SSE}/\text{SST}$ follows straight from it.

B3. 95%
A model that always predicts the majority class is right exactly as often as that class occurs — here 95%. This is why accuracy is close to useless on imbalanced data: it sets a floor that looks impressive and is achieved by a model that has learned nothing and never finds a single positive case. Its recall is 0 and its precision is undefined. Quote precision, recall and F_1 for the rare class instead, and always state what the always-negative baseline would have scored.

B4. reinforcement
Supervised learning has labelled examples; unsupervised has none and must find structure in the inputs alone; *reinforcement* learning has neither, and instead learns a policy from reward collected by acting.

Part C — True or false

$3 \times 1 = 3$ marks

The statement is reproduced first, then the answer and why. A reason was optional on the paper, but writing one is how you find out whether you actually had the idea.

C1. *Between target leakage and preprocessing leakage, a `Pipeline` prevents both.*

False, and the distinction is worth holding onto. A `Pipeline` prevents *preprocessing* leakage — it makes sure every fitted step (scaler, imputer, encoder, selector, resampler) sees training rows only. It does nothing whatever about *target* leakage, which is a feature that would not exist at prediction time or is computed from the answer. No amount of pipeline discipline rescues you from a column that is secretly the target; only understanding what your columns mean does.

C2. *Between training R^2 and held-out R^2, neither can be negative.*

False. On held-out data the baseline is the *test* mean, and a model can be worse than simply predicting it — in which case SSE > SST and $R^2 = 1 - \text{SSE}/\text{SST}$ goes negative. The $0 \leq R^2 \leq 1$ range you remember holds for the *training* fit of a model with an intercept, and nowhere else. A negative held-out R^2 is a real and useful signal: it says the model is not merely weak but actively worse than a constant.

C3. *Between cross-entropy and accuracy, only the first can be the quantity a gradient method minimises.*

True. Accuracy counts how many predictions fall on the right side of a threshold, so as a function of the parameters it is a step function — flat almost everywhere, with jumps where a prediction crosses over. Its gradient is zero wherever it exists, so gradient descent has nothing to descend. That is exactly why we train on a smooth surrogate such as cross-entropy and only *report* accuracy.

Part D — Short answer $2 \times 2 = 4$ marks

D1. Compare the four points $(1, 10)$, $(3, 20)$, $(5, 32)$, $(7, 46)$ with the least-squares line through them. Fit $\hat{y} = b_0 + b_1 x$, using $b_1 = S_{xy}/S_{xx}$ and $b_0 = \bar{y} - b_1 \bar{x}$.

$\bar{x} = 4$ and $\bar{y} = 27$. Building the two sums from the deviations:

$$S_{xx} = \sum (x_i - \bar{x})^2 = \mathbf{20}, \qquad S_{xy} = \sum (x_i - \bar{x})(y_i - \bar{y}) = \mathbf{120}$$

$$b_1 = \frac{S_{xy}}{S_{xx}} = \frac{120}{20} = \mathbf{6}, \qquad b_0 = \bar{y} - b_1 \bar{x} = 27 - (6)(4) = \mathbf{3}$$

So the fitted line is $\hat{y} = 3 + 6x$. Its residuals are $+1$, -1, -1, $+1$, which sum to zero — as they must.

Why $\sum e_i$ had to be zero. You did not need the residuals to know it. Setting $\partial S/\partial b_0 = 0$ gives $-2\sum(y_i - b_0 - b_1 x_i) = 0$, and the bracket *is* the residual — so the first normal equation says exactly $\sum e_i = 0$. The second, $\partial S/\partial b_1 = 0$, says $\sum x_i e_i = 0$. Together they are worth carrying as checks: if your residuals do not sum to zero, the arithmetic is wrong somewhere. They also force the line through $(\bar{x}, \bar{y})$, and make SST = SSR + SSE exact rather than approximate — here $724 = 720 + 4$.

D2. Compare two loss curves from the same model: one falls and flattens onto a level, the other falls and then bounces up and down about a level without settling.

(a) Two causes both produce a loss that falls and then bounces without settling.

The first is a **learning rate slightly too large**: the step overshoots the minimum, comes back, overshoots again, and the iterate circles a basin it never lands in. The second is the **gradient-noise floor**: with a small batch and a constant step size, each update is computed from a noisy estimate of the gradient, and once the noise is the same size as the signal the iterate rattles about the optimum instead of converging to it. A third acceptable answer is a schedule that simply needed decaying.

(b) They look identical on the curve, so change one thing and watch. **Lower η (or raise the batch size) and re-run.** If the curve now settles, the step size was the problem. If it still bounces at about the same level, you are at the noise floor — and the fix is different: decay η over time, or average more examples per update. Distinguishing two hypotheses by a single change you can actually make is the whole method here.

Part E — Reasoning

4 marks

E1. Two colleagues run the same six steps in different orders, and get very different test scores. One of the features, `price_per_sqft`, was computed as Price/Area, and Price is the target.

(1) Standardise the numeric columns. **(2)** Split into training and test sets.
(3) Median-impute the missing areas. **(4)** Drop the `ID` column.
(5) Oversample the minority class. **(6)** Fit the model.

(i) A safe order: 4, 2, 3, 1, 5, 6. Drop `ID` first — it is an identifier and carries no signal. Then **split**. Then, on the training part only, impute the missing areas, standardise, and oversample the minority class. Fit last. Any order works provided the split comes before all three fitted steps and the model is fitted at the end.

(ii) The split. Every step after it learns something *from* the data — a median, a mean and standard deviation, which rows to duplicate. Put any of them before the split and that statistic has been computed partly from the rows you meant to hold back, so the test score is measuring a model that has already seen them. Oversampling before the split is the worst of the three: duplicated minority rows end up in both halves, so the model is tested on rows it trained on.

(iii) `price_per_sqft`. It is computed as price divided by area, and price is the target — so the column hands the model the answer. This is *target* leakage, and unlike the others it does not depend on *where* the step sits. Reorder the pipeline all you like and the column still contains the answer. A `Pipeline` governs what is fitted and when; it has no opinion about what a column means. The test is: *at the moment I make this prediction, would I actually have this value?* Here you would not — you would not know the price.

(iv) Dropping `ID`. It is safe before the split because it fits nothing. Imputing, scaling and oversampling all estimate something from the rows they see; dropping a column is a decision about which columns exist, taken from domain knowledge rather than from the data's values. The same is true of correcting a known data-entry error. Nothing crosses from test to train, so nothing leaks.

Questions on any of this are welcome in the next class or in office hours.

CSAI2017P — Applied Machine Learning (Theory)
Theory Quiz 4 — Solutions • Set H

Maximum marks **20** • 15 questions on four pages

These are the answers to **Set H**, with the reasoning behind each one. Find the set letter on your own paper and work from the matching four pages. Sets were handed out at random and every set covers the same material, so if you want the extra practice the other sets are worth reading too.
Several of these questions have more than one good route to the answer, and a route different from the one printed here is not automatically a worse one. If you reached a different answer and can show your reasoning, bring your script and let us go through it.

Part A — Multiple choice

$5 \times 1 = 5$ marks

A1. (a) the training rows only
The same principle as the worked line: anything *fitted* to data must be fitted to the training rows alone.

A2. (c) unaffected by scaling
The contrast is the point. Distance mixes the columns, so scale matters; a threshold on one column does not, so it does not.

A3. (d) among those at right angles to the first
Each component takes the most variance still available, subject to being orthogonal to the ones already chosen.

A4. (b) η is past $2/L$
Flattening is health; overflowing is a step size past the threshold.

A5. (c) unsupervised learning
Labelled continuous target is supervised regression; unlabelled groups is unsupervised.

Part B — Fill in the blanks

$4 \times 1 = 4$ marks

B1. S_{xx}
The slope is $b_1 = S_{xy}/S_{xx}$, where $S_{xx} = \sum(x_i - \bar{x})^2$ measures how spread out the x values are and $S_{xy} = \sum(x_i - \bar{x})(y_i - \bar{y})$ measures how the two move together. Dividing one by the other converts "they move together this much" into "y rises this much per unit of x". It is defined only when $S_{xx} > 0$ — if every x were the same there would be no line to fit.

B2. SSE
SST = SSR + SSE, and this is an *exact* identity rather than an approximation. It holds because of the normal equations: the residuals come out orthogonal to the fitted values, so the cross-term vanishes. SST is the variation you started with, SSR the part the line accounts for, SSE the part it does not — and $R^2 = \text{SSR}/\text{SST} = 1 - \text{SSE}/\text{SST}$ follows straight from it.

B3. 95%
A model that always predicts the majority class is right exactly as often as that class occurs — here 95%. This is why accuracy is close to useless on imbalanced data: it sets a floor that looks impressive and is achieved by a model that has learned nothing and never finds a single positive case. Its recall is 0 and its precision is undefined. Quote precision, recall and F_1 for the rare class instead, and always state what the always-negative baseline would have scored.

B4. reinforcement
Supervised learning has labelled examples; unsupervised has none and must find structure in the inputs alone; *reinforcement* learning has neither, and instead learns a policy from reward collected by acting.

Part C — True or false

$3 \times 1 = 3$ marks

The statement is reproduced first, then the answer and why. A reason was optional on the paper, but writing one is how you find out whether you actually had the idea.

C1. A `Pipeline` stops a scaler seeing the test rows. *It therefore also stops a feature computed from the target being used.*

False, and the distinction is worth holding onto. A `Pipeline` prevents *preprocessing* leakage — it makes sure every fitted step (scaler, imputer, encoder, selector, resampler) sees training rows only. It does nothing whatever about *target* leakage, which is a feature that would not exist at prediction time or is computed from the answer. No amount of pipeline discipline rescues you from a column that is secretly the target; only understanding what your columns mean does.

C2. Training R^2 lies between 0 and 1. *Held-out R^2 is therefore also bounded below by* 0.

False. On held-out data the baseline is the *test* mean, and a model can be worse than simply predicting it — in which case SSE > SST and $R^2 = 1 - \text{SSE/SST}$ goes negative. The $0 \leq R^2 \leq 1$ range you remember holds for the *training* fit of a model with an intercept, and nowhere else. A negative held-out R^2 is a real and useful signal: it says the model is not merely weak but actively worse than a constant.

C3. Cross-entropy is smooth, so gradient descent can minimise it. *Accuracy is a step function, so it cannot.*

True. Accuracy counts how many predictions fall on the right side of a threshold, so as a function of the parameters it is a step function — flat almost everywhere, with jumps where a prediction crosses over. Its gradient is zero wherever it exists, so gradient descent has nothing to descend. That is exactly why we train on a smooth surrogate such as cross-entropy and only *report* accuracy.

Part D — Short answer $2 \times 2 = 4$ marks

D1. *For* $(0,1)$ *and* $(2,5)$ *the slope is* $S_{xy}/S_{xx} = 4/2 = 2$ *and the intercept is* $3 - 2(1) = 1$. Now fit $\hat{y} = b_0 + b_1 x$ to $(1,18)$, $(3,10)$, $(5,4)$, $(7,0)$ by the same route.

$\bar{x} = 4$ and $\bar{y} = 8$. Building the two sums from the deviations:

$$S_{xx} = \sum (x_i - \bar{x})^2 = \mathbf{20}, \qquad S_{xy} = \sum (x_i - \bar{x})(y_i - \bar{y}) = \mathbf{-60}$$

$$b_1 = \frac{S_{xy}}{S_{xx}} = \frac{-60}{20} = \mathbf{-3}, \qquad b_0 = \bar{y} - b_1 \bar{x} = 8 - (-3)(4) = \mathbf{20}$$

So the fitted line is $\hat{y} = 20 - 3x$. Its residuals are $+1$, -1, -1, $+1$, which sum to zero — as they must.

Why $\sum e_i$ had to be zero. You did not need the residuals to know it. Setting $\partial S/\partial b_0 = 0$ gives $-2\sum(y_i - b_0 - b_1 x_i) = 0$, and the bracket *is* the residual — so the first normal equation says exactly $\sum e_i = 0$. The second, $\partial S/\partial b_1 = 0$, says $\sum x_i e_i = 0$. Together they are worth carrying as checks: if your residuals do not sum to zero, the arithmetic is wrong somewhere. They also force the line through $(\bar{x}, \bar{y})$, and make SST = SSR + SSE exact rather than approximate — here $184 = 180 + 4$.

D2. *A loss that falls and then flattens onto a level has converged.* A loss that falls and then bounces up and down about a level, never settling, has not.

(a) Two causes both produce a loss that falls and then bounces without settling.

The first is a **learning rate slightly too large**: the step overshoots the minimum, comes back, overshoots again, and the iterate circles a basin it never lands in. The second is the **gradient-noise floor**: with a small batch and a constant step size, each update is computed from a noisy estimate of the gradient, and once the noise is the same size as the signal the iterate rattles about the optimum instead of converging to it. A third acceptable answer is a schedule that simply needed decaying.

(b) They look identical on the curve, so change one thing and watch. **Lower η (or raise the batch size) and re-run.** If the curve now settles, the step size was the problem. If it still bounces at about the same level, you are at the noise floor — and the fix is different: decay η over time, or average more examples per update. Distinguishing two hypotheses by a single change you can actually make is the whole method here.

Part E — Reasoning

4 marks

E1. *Worked analogue. A scaler fitted before the split leaks, because the test rows helped compute the mean. Moving it after the split fixes it. But a column that is secretly the answer leaks no matter where it sits — moving it changes nothing.*

Now this case. The six steps below are in no particular order, and one feature, `price_per_sqft`, was computed as Price/Area, where Price is the target.

(1) Standardise the numeric columns. **(2)** Split into training and test sets.
(3) Median-impute the missing areas. **(4)** Drop the `ID` column.
(5) Oversample the minority class. **(6)** Fit the model.

(i) A safe order: 4, 2, 3, 1, 5, 6. Drop `ID` first — it is an identifier and carries no signal. Then **split**. Then, on the training part only, impute the missing areas, standardise, and oversample the minority class. Fit last. Any order works provided the split comes before all three fitted steps and the model is fitted at the end.

(ii) The split. Every step after it learns something *from* the data — a median, a mean and standard deviation, which rows to duplicate. Put any of them before the split and that statistic has been computed partly from the rows you meant to hold back, so the test score is measuring a model that has already seen them. Oversampling before the split is the worst of the three: duplicated minority rows end up in both halves, so the model is tested on rows it trained on.

(iii) `price_per_sqft`. It is computed as price divided by area, and price is the target — so the column hands the model the answer. This is *target* leakage, and unlike the others it does not depend on *where* the step sits. Reorder the pipeline all you like and the column still contains the answer. A `Pipeline` governs what is fitted and when; it has no opinion about what a column means. The test is: *at the moment I make this prediction, would I actually have this value?* Here you would not — you would not know the price.

(iv) Dropping `ID`. It is safe before the split because it fits nothing. Imputing, scaling and oversampling all estimate something from the rows they see; dropping a column is a decision about which columns exist, taken from domain knowledge rather than from the data's values. The same is true of correcting a known data-entry error. Nothing crosses from test to train, so nothing leaks.

Questions on any of this are welcome in the next class or in office hours.